

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук

должность, уч. степень, звание

подпись, дата

А. В. Бржезовский

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 8

КУРСОРЫ

по курсу:

МЕТОДЫ И СРЕДСТВА ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: изучение механизмов курсоров в MySQL и их использования для последовательной обработки, изменения и удаления записей.

2 Постановка задачи

Требуется реализовать процедуры с использованием курсоров, продемонстрировать особенности работы курсоров в MySQL, а также реализовать изменение и удаление данных при последовательной обработке записей.

Вариант: 4.

Создайте базу данных для хранения следующих сведений: студент, группа, дисциплина, преподаватель, лабораторная работа, рейтинг за сданную лабораторную работу.

3 Ход выполнения

В ходе выполнения работы была использована ранее созданная база данных, содержащая сведения о студентах, группах, дисциплинах, лабораторных работах и оценках.

На первом этапе была реализована хранимая процедура с использованием курсора. Курсор последовательно перебирает оценки студентов по дисциплине «Базы данных» и автоматически повышает неудовлетворительные оценки до оценки «3».

На рисунке 1 приведен код процедуры с курсором и результат ее выполнения.

```

27
28 ● -- курсор для обновления оценок
29 CREATE PROCEDURE fix_low_scores()
30 BEGIN
31     DECLARE done INT DEFAULT 0;
32     DECLARE v_rating_id INT;
33     DECLARE v_score INT;
34
35 ● DECLARE cur CURSOR FOR
36 SELECT rating_id, score
37 FROM lab_rating;
38
39 DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
40
41 OPEN cur;
42
43 read_loop: LOOP
44     FETCH cur INTO v_rating_id, v_score;
45
46 ● IF done THEN
47     LEAVE read_loop;
48 END IF;
49
50 ● IF v_score < 3 THEN
51     UPDATE lab_rating
52     SET score = 3
53     WHERE rating_id = v_rating_id;
54 END IF;
55
56 END LOOP;
57
58 ● CLOSE cur;
59 END //
60
61 DELIMITER ;
62
63 ● -- запуск обновления
64 CALL fix_low_scores();
65
66 ● -- просмотр результата
67 SELECT * FROM lab_rating;
68
69

```

lab_rating 1 X lab_rating 1 (2) Statistics 1

SELECT * FROM lab_rating | Enter a SQL expression to filter results (use Ctrl+Space)

	123 rating_id	123 student_id	123 lab_id	123 score	submit
7	36	73	1	3	
8	37	73	2	3	
9	38	73	4	3	
10	39	18	1	3	
11	40	18	2	3	

Рисунок 1 — Код процедуры с курсором и ее выполнение. В результате оценки со значением менее 3 были заменены на «3»

Далее была реализована процедура удаления записей с использованием курсора. В процессе работы курсор последовательно перебирает оценки студентов и удаляет записи с нулевым рейтингом.

На рисунке 2 приведен код процедуры удаления записей посредством курсора и результат ее выполнения.

```

72  -- курсор для удаления нулевых оценок
73  CREATE PROCEDURE delete_zero_scores()
74  BEGIN
75      DECLARE done INT DEFAULT 0;
76      DECLARE v_rating_id INT;
77      DECLARE v_score INT;
78
79      DECLARE cur CURSOR FOR
80      SELECT rating_id, score
81      FROM lab_rating;
82
83      DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
84
85      OPEN cur;
86
87      delete_loop: LOOP
88
89          FETCH cur INTO v_rating_id, v_score;
90
91          IF done THEN
92              LEAVE delete_loop;
93          END IF;
94
95          IF v_score = 0 THEN
96              DELETE FROM lab_rating
97              WHERE rating_id = v_rating_id;
98          END IF;
99
100         END LOOP;
101
102     CLOSE cur;
103 END //
104
105 DELIMITER ;
106
107 INSERT INTO lab_rating
108 (student_id, lab_id, score, submission_date)
109 VALUES (22, 1, 0, '2026-01-10'),
110         (22, 2, 0, '2026-01-10'),
111         (22, 3, 0, '2026-01-10');
112
113 -- запуск удаления
114 CALL delete_zero_scores();
115
116 -- просмотр результата
117 SELECT * FROM lab_rating;
118

```

lab_rating 1
lab_rating 1 (2) X
Statistics 1

SELECT * FROM lab_rating
Enter a SQL expression to filter results (use Ctrl+Space)

	123 rating_id	123 student_id	123 lab_id	123 score	123 submission_date
9	38	73	4	3	2026
10	39	18	1	3	2026
11	40	18	2	3	2026

Рисунок 2 — Код процедуры с курсором на удаление. Перед выполнением были добавлены записи с нулями, в результате строки с оценками «0» были удалены из таблицы

Также было исследовано различие между статическими, динамическими курсорами и курсорами KEYSET: первый тип фиксирует набор данных единожды и не чувствителен к сторонним изменениям, пока курсор открыт, второй тип, наоборот, чувствителен к изменениям, а третий сохраняет набор ключей исходного запроса и чувствителен к изменениям значений в рамках сохраненного набора ключей. В ходе анализа было установлено, что MySQL

поддерживает только серверные курсоры, реализованные внутри хранимых процедур, и не поддерживает курсоры типов STATIC, DYNAMIC и KEYSET, а также синтаксис Transact-SQL Extended Syntax и конструкцию WHERE CURRENT OF. В связи с этим обработка записей была реализована средствами курсоров MySQL с использованием FETCH и последующих SQL-операторов UPDATE и DELETE. Созданный SQL-скрипт приведен в Приложении.

4 Выводы

В ходе выполнения работы были изучены механизмы курсоров в реляционной базе данных. Были реализованы процедуры последовательной обработки записей с использованием курсоров и операций FETCH.

С помощью курсоров были реализованы операции изменения и удаления данных. Также были исследованы ограничения реализации курсоров в MySQL. Установлено, что MySQL не поддерживает курсоры типов STATIC, DYNAMIC и KEYSET, а также конструкцию WHERE CURRENT OF, поэтому обработка записей выполняется через выборку идентификаторов и последующее выполнение SQL-операторов UPDATE и DELETE.

Получен практический опыт использования курсоров для построчной обработки данных внутри хранимых процедур.

ПРИЛОЖЕНИЕ

```
USE university_db;

-- тестовые данные
INSERT INTO lab_rating(student_id, lab_id, score, submission_date)
SELECT 18, 1, 2, CURDATE()
WHERE NOT EXISTS (
    SELECT 1
    FROM lab_rating
    WHERE student_id = 18
    AND lab_id = 1
);

INSERT INTO lab_rating(student_id, lab_id, score, submission_date)
SELECT 18, 2, 0, CURDATE()
WHERE NOT EXISTS (
    SELECT 1
    FROM lab_rating
    WHERE student_id = 18
    AND lab_id = 2
);

UPDATE lab_rating
SET score = 2
WHERE score = 3;

DELIMITER //

-- курсор для обновления оценок
CREATE PROCEDURE fix_low_scores()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_rating_id INT;
    DECLARE v_score INT;

    DECLARE cur CURSOR FOR
    SELECT rating_id, score
    FROM lab_rating;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN cur;

    read_loop: LOOP

        FETCH cur INTO v_rating_id, v_score;

        IF done THEN
            LEAVE read_loop;
        END IF;

        IF v_score < 3 THEN
            UPDATE lab_rating
            SET score = 3
            WHERE rating_id = v_rating_id;
        END IF;

    END LOOP;

    CLOSE cur;
END //
```

```

DELIMITER ;

-- запуск обновления
CALL fix_low_scores();

-- просмотр результата
SELECT * FROM lab_rating;

DELIMITER //

-- курсор для удаления нулевых оценок
CREATE PROCEDURE delete_zero_scores()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_rating_id INT;
    DECLARE v_score INT;

    DECLARE cur CURSOR FOR
    SELECT rating_id, score
    FROM lab_rating;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN cur;

    delete_loop: LOOP

        FETCH cur INTO v_rating_id, v_score;

        IF done THEN
            LEAVE delete_loop;
        END IF;

        IF v_score = 0 THEN
            DELETE FROM lab_rating
            WHERE rating_id = v_rating_id;
        END IF;

    END LOOP;

    CLOSE cur;
END //

DELIMITER ;

INSERT INTO lab_rating
(student_id, lab_id, score, submission_date)
VALUES (22, 1, 0, '2026-01-10'),
(22, 2, 0, '2026-01-10'),
(22, 3, 0, '2026-01-10');

-- запуск удаления
CALL delete_zero_scores();

-- просмотр результата
SELECT * FROM lab_rating;

```